SARIMA

Daniel de Freitas

Instituto J&F
Centro de Educação para os Negócios

SARIMA

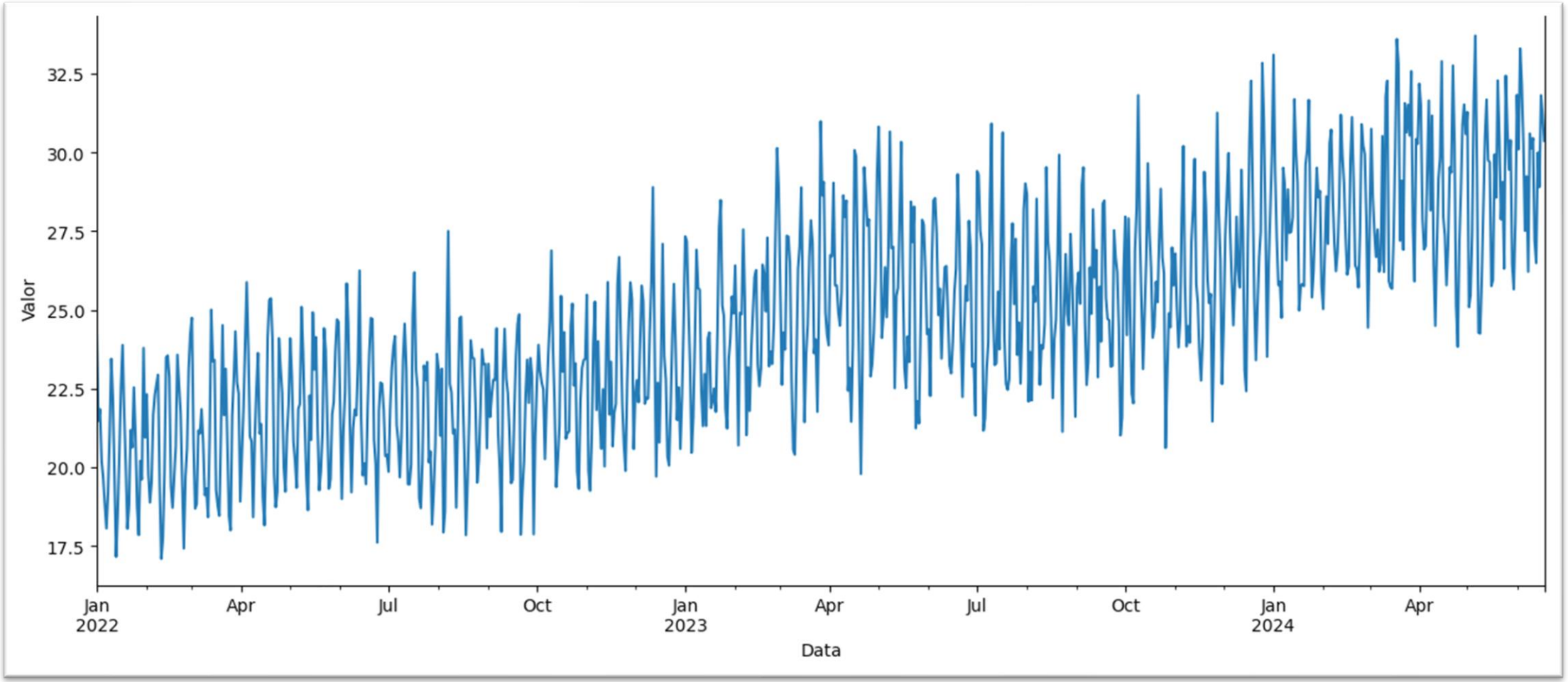
O SARIMA $(p,d,q)\times(P,D,Q)m$ modela a série em camadas:

$$\underbrace{\Phi_P(B^m)\,\phi_p(B)}_{\text{AR sazonal e não-sazonal}}\ \underbrace{(1-B)^d\,(1-B^m)^D}_{\text{diferenças}}\ y_t \;=\; \underbrace{\Theta_Q(B^m)\,\theta_q(B)}_{\text{MA sazonal e não-sazonal}}\ \varepsilon_t$$

- d: quantas **diferenças não sazonais** para remover **tendência**.
- D: quantas **diferenças sazonais** (a cada m) para remover **sazonalidade persistente**.
- p: quantos **valores passados** (lags 1..p) entram linearmente (memória curta).
- q: quantos **erros passados** (lags 1..q) entram (suaviza choques de curto prazo).
- P: quantos **valores sazonais passados** (lags $m, 2m, \ldots$) entram.
- Q: quantos **erros sazonais passados** entram (captura choques que se repetem por estação).
- m: **período sazonal** (ex.: diário com ciclo semanal → $m = 7$; mensal com ciclo anual → $m = 12$).

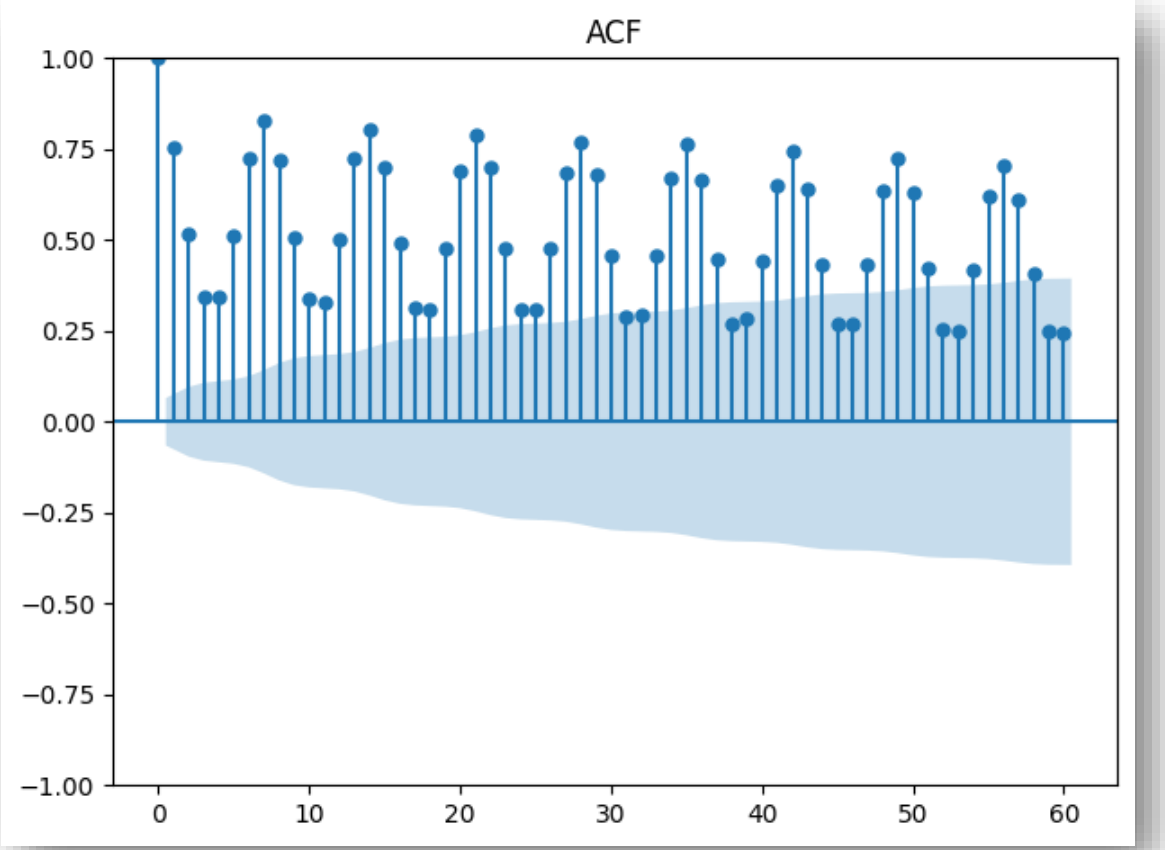
Serie:

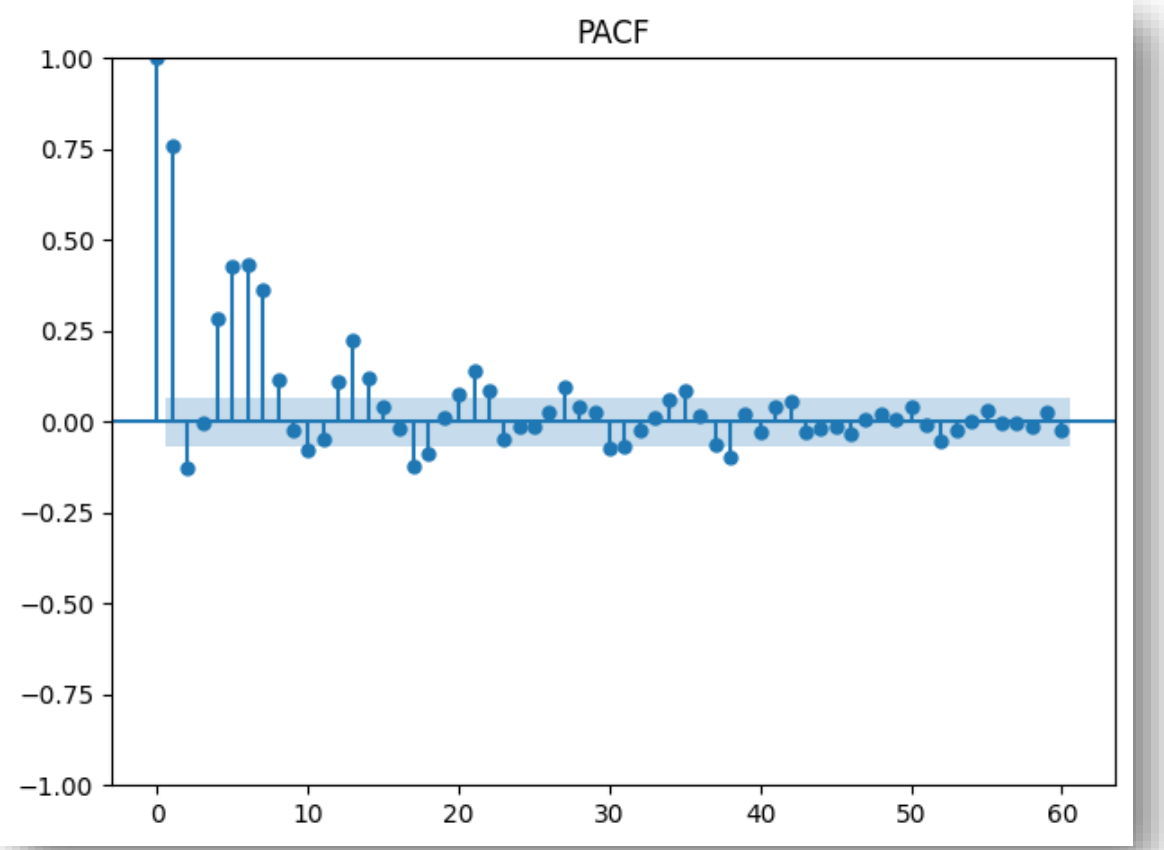
Considere a série presente no arquivo **“serie_sarima.xlsx”** :



ACF, PACF e ADF KPSS:

ACF, PACF e ADF KPSS:






```
ADF_pvalue      0.898933
KPSS_pvalue     0.010000
dtype: float64
```

STL

STL:

Seasonal-Trend decomposition using Loess decompõem a série em:

$$y_t = \boldsymbol{Tendência_t} + \boldsymbol{Sazonalidade_t} + \boldsymbol{Ruído_t}$$

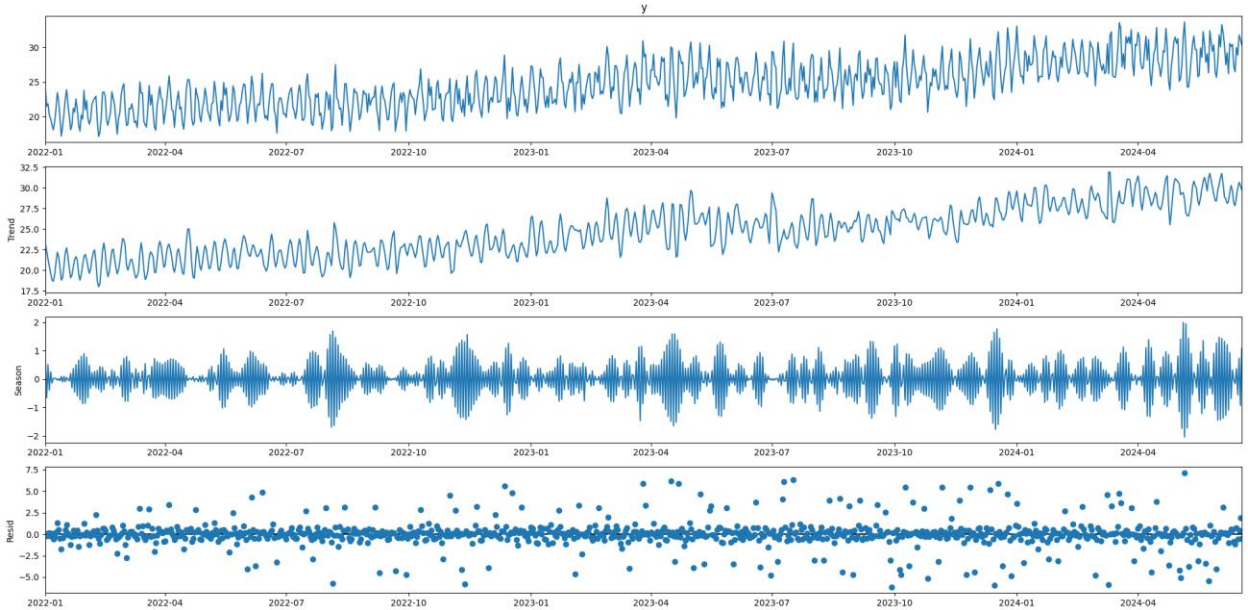
```
from statsmodels.tsa.seasonal import STL
```

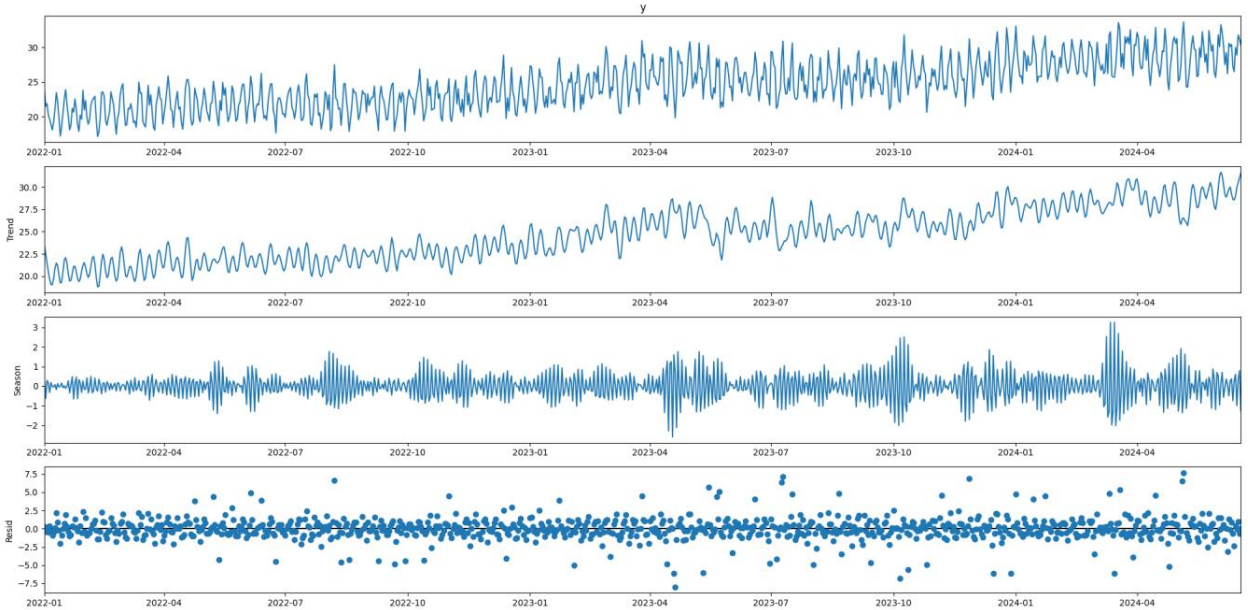
STL:

Devemos tentar entender se há sazonalidade, alterando o valor de m (período)

```
m = 2
stl = STL(ts, period=m, robust=True).fit()
fig = stl.plot()
fig.set_size_inches(20,10)
plt.tight_layout()
plt.show()
```

```
m = 3
stl = STL(ts, period=m, robust=True).fit()
fig = stl.plot()
fig.set_size_inches(20,10)
plt.tight_layout()
plt.show()
```





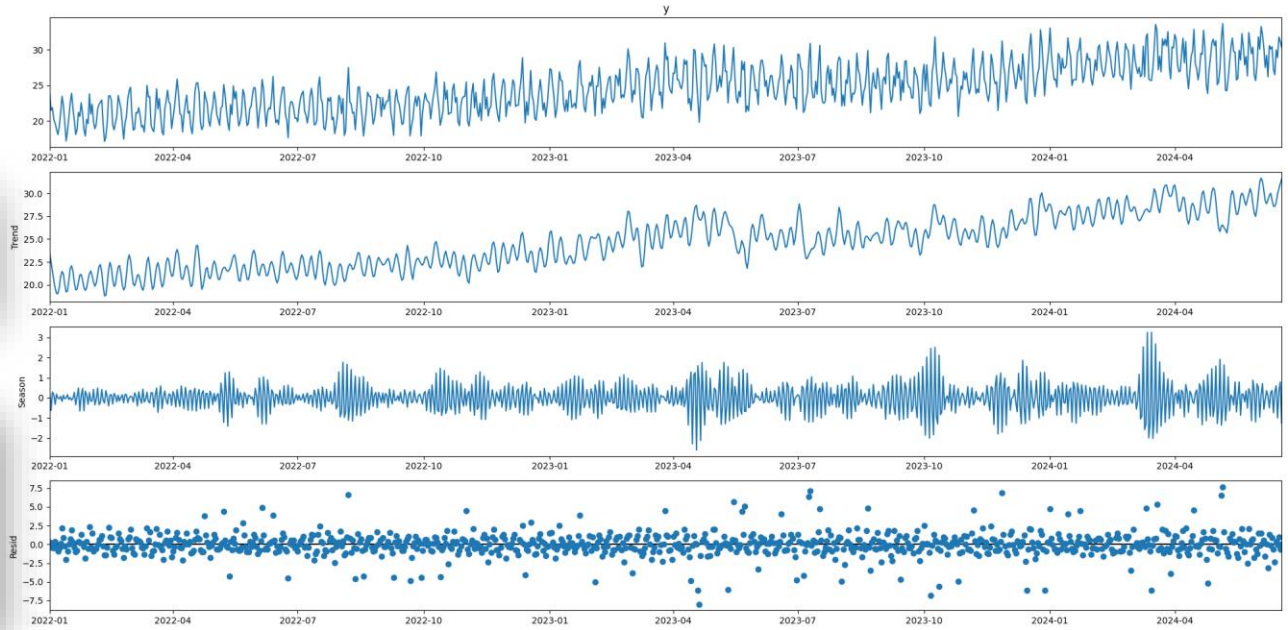
Força da Sazonalidade:

$$\textbf{Força da sazonalidade } (F_S) = \max\left(0,\ 1 - \frac{\mathrm{Var}(R_t)}{\mathrm{Var}(R_t + S_t)}\right)$$

```
forca_sazonalidade = 1 - (np.var(stl.resid) / np.var(stl.seasonal + stl.resid))
print(f"Força de sazonalidade (0-1): {forca_sazonalidade:.4f}")
```

```
Força de sazonalidade (0-1): 0.0195
```

m = 3



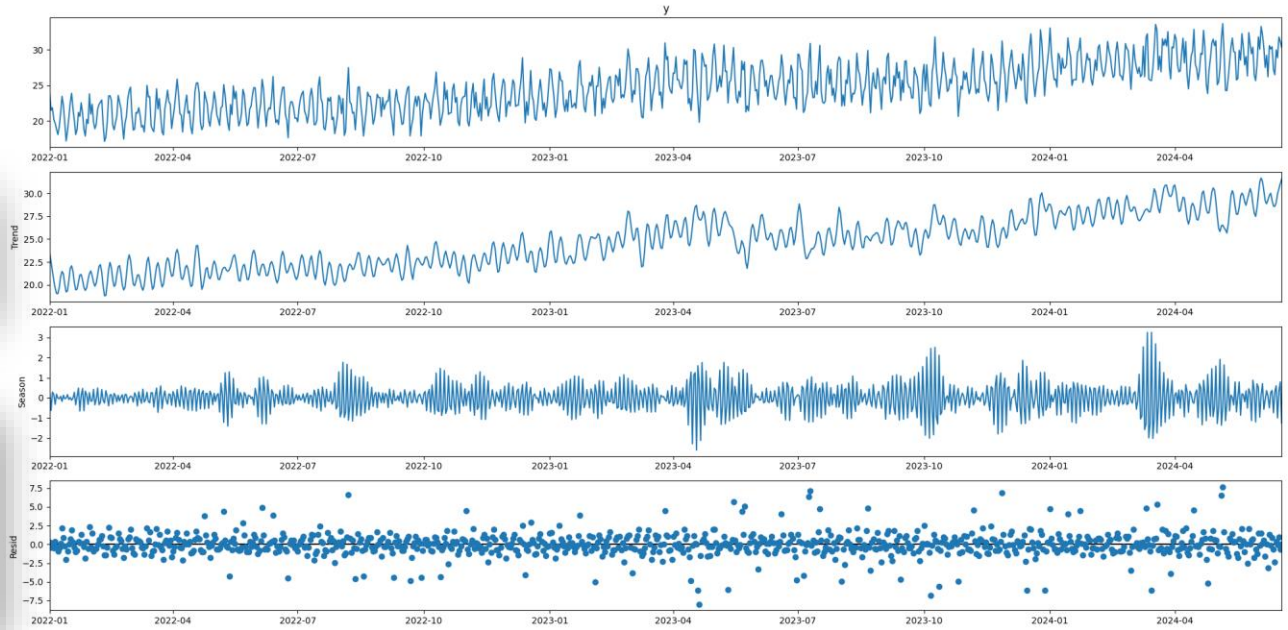
Força da Sazonalidade:

m = 3

$$\textbf{Força da sazonalidade } (F_S) = \max\left(0,\ 1 - \frac{\mathrm{Var}(R_t)}{\mathrm{Var}(R_t + S_t)}\right)$$

```python
forca_sazonalidade = 1 - (np.var(stl.resid) / np.var(stl.seasonal + stl.resid))
print(f"Força de sazonalidade (0-1): {forca_sazonalidade:.4f}")
```

```
Força de sazonalidade (0-1): 0.0195
```

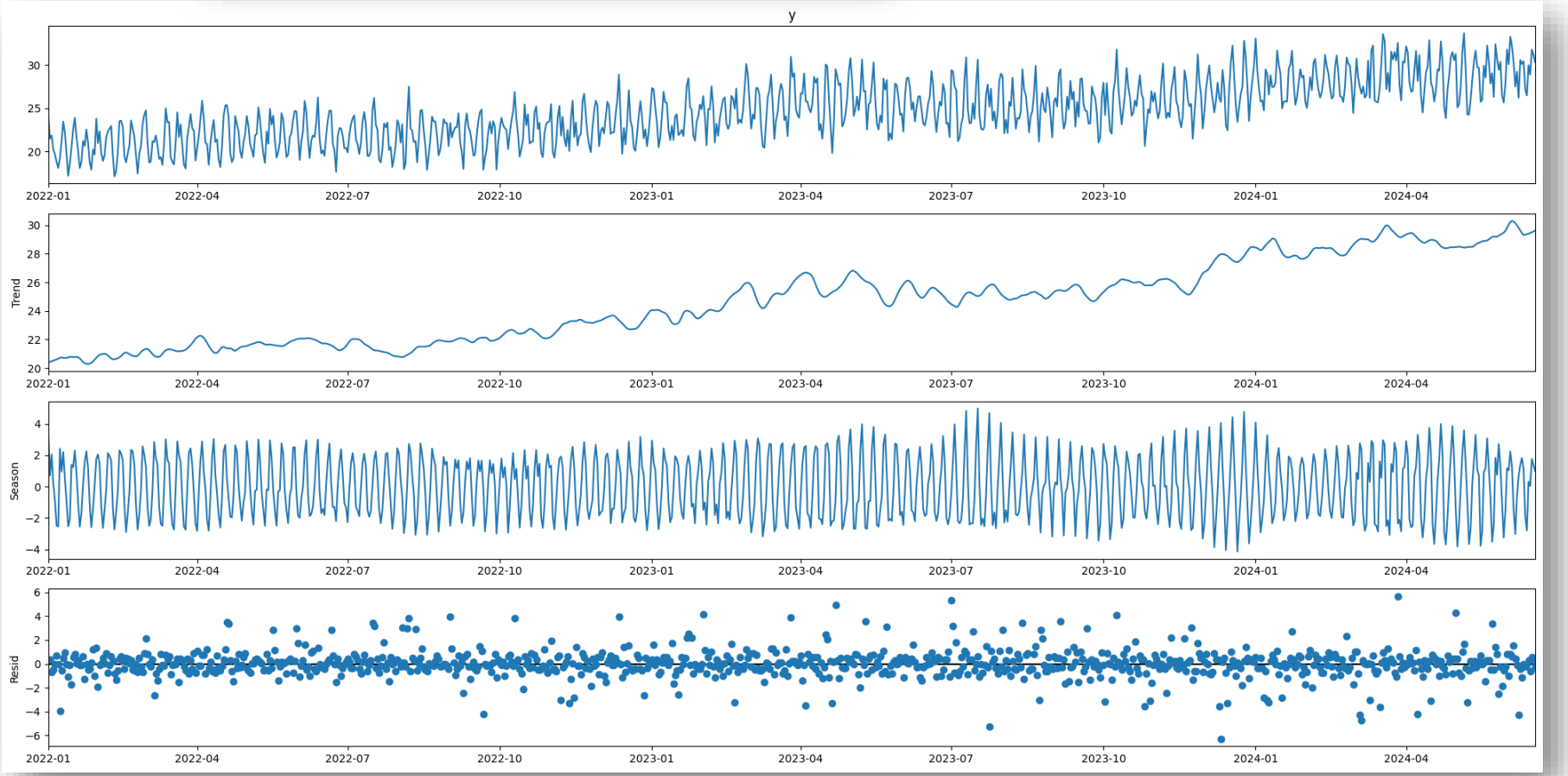


Vamos tentar encontrar o m com maior força de sazonalidade !!

Força da Sazonalidade:

m = 7

```
Força de sazonalidade (0-1): 0.7062
```



SARIMA (p,d,q)x(P,D,Q,m)

Função e espaço de busca

```python
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```python
# treino teste
h = 10
ytrain = ts.iloc[:-h]
ytest  = ts.iloc[-h:]

# espaco de busca
m = 7
p_range = [0,1,2]
d_range = [0,1]
q_range = [0,1,2]
P_range = [0,1,2]
D_range = [0,1]
Q_range = [0,1,2]

def tentar_sarima(order, seasonal_order, y):
    try:
        model = SARIMAX(y, order=order, seasonal_order=seasonal_order,
                        enforce_stationarity=False, enforce_invertibility=False)
        res = model.fit(disp=False, maxiter=100)
        return res
    except Exception:
        return None
```

- Definimos horizonte
- Definimos range de busca
- Para facilitar o “grid” fazemos uma função
- **order** e **seasonal order** são, respectivamente, as tuplas **(p,d,q)** e **(P,D,Q,m)**
- Os parâmetros enforce são para flexibilizar o otimizador a convergir
- disp=False não imprime os logs

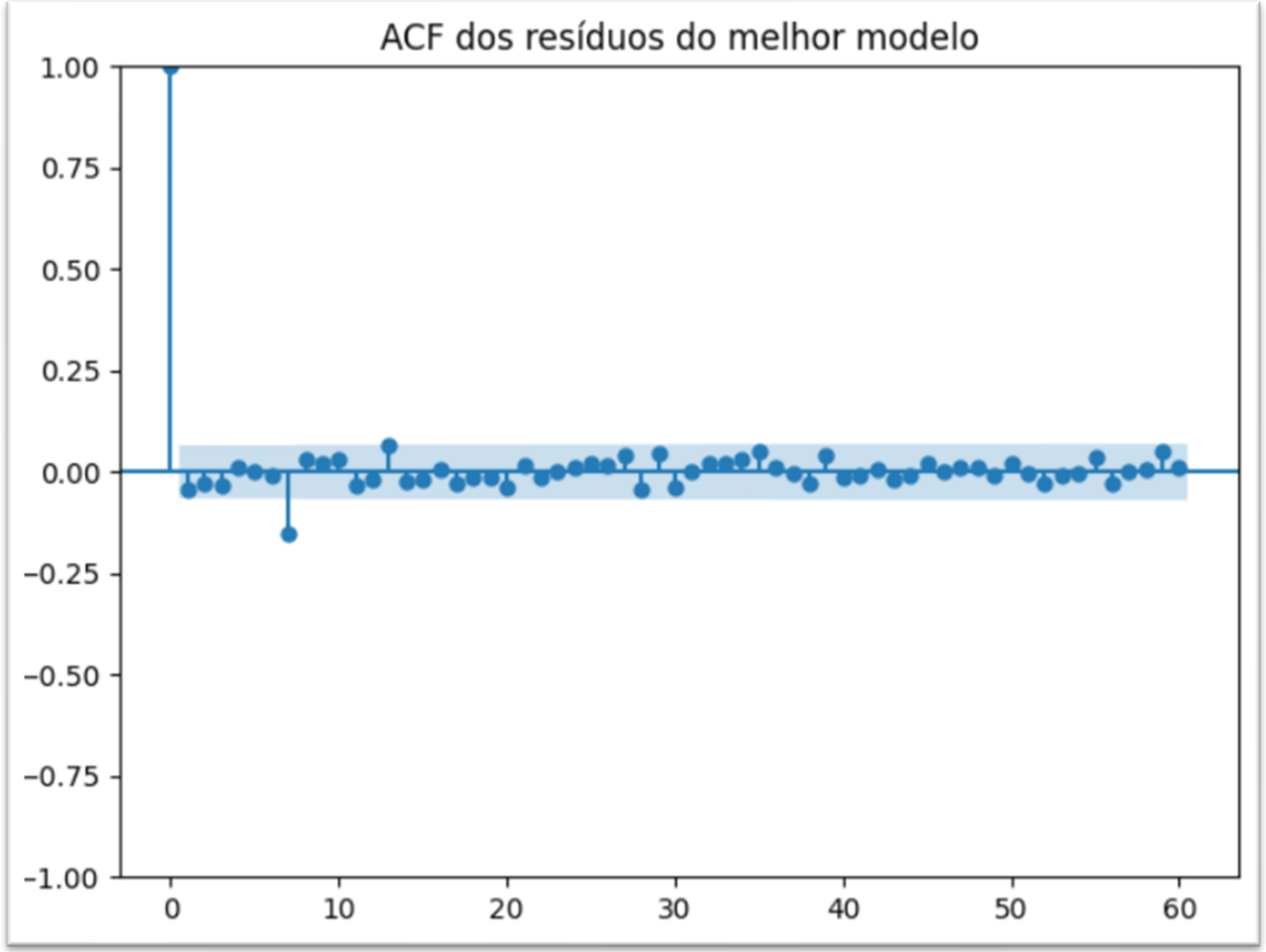
Resultado

```
        order seasonal_order          BIC
0  (0, 1, 2)   (0, 1, 2, 7)  3131.608242
1  (0, 1, 2)   (1, 1, 2, 7)  3133.104160
2  (1, 1, 1)   (0, 1, 2, 7)  3135.923687
3  (0, 1, 1)   (0, 1, 2, 7)  3136.756582
4  (1, 1, 2)   (0, 1, 2, 7)  3137.313076
5  (1, 1, 2)   (1, 1, 2, 7)  3139.065061
6  (1, 1, 1)   (1, 1, 2, 7)  3139.544890
7  (0, 1, 1)   (1, 1, 2, 7)  3140.200690
8  (2, 1, 1)   (0, 1, 2, 7)  3140.577569
9  (1, 0, 2)   (0, 1, 2, 7)  3141.521071

Melhor por BIC: (0, 1, 2) x (0, 1, 2, 7)  | BIC = 3131.61
```

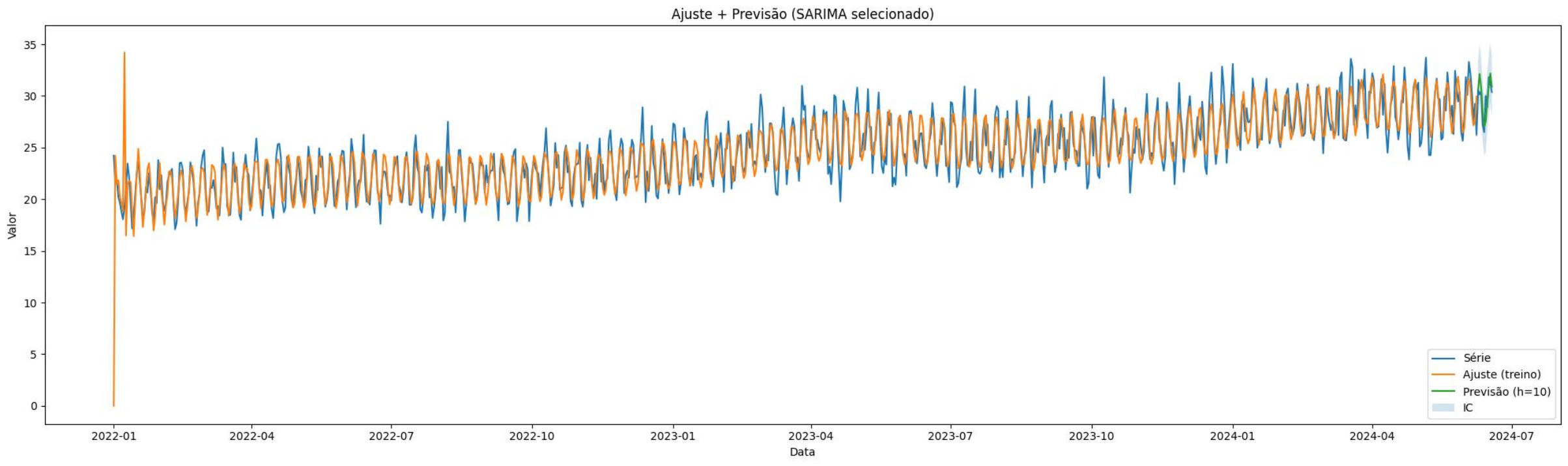
- Encontramos o melhor modelo nessa combinação: **SARIMA (0,1,2)x (0,1,2,7)**

Análise de resíduos:



	lb_stat	lb_pvalue
14	31.340703	0.004966
21	34.348681	0.033243

Previsão:

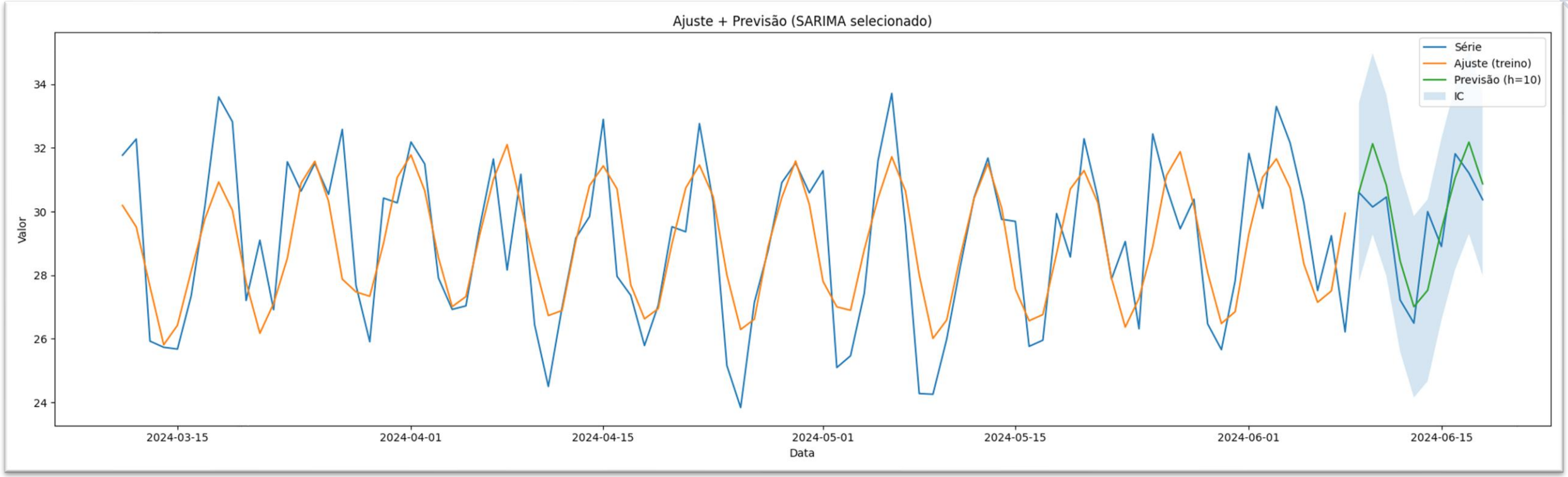

Ajuste + Previsão (SARIMA selecionado)
Valor
Data
35
30
25
20
15
10
5
0
2022-01
2022-04
2022-07
2022-10
2023-01
2023-04
2023-07
2023-10
2024-01
2024-04
2024-07
Série
Ajuste (treino)
Previsão (h=10)
IC

Previsão:

```
MAE  (h=10): 0.932
MAPE (h=10): 3.15%
```

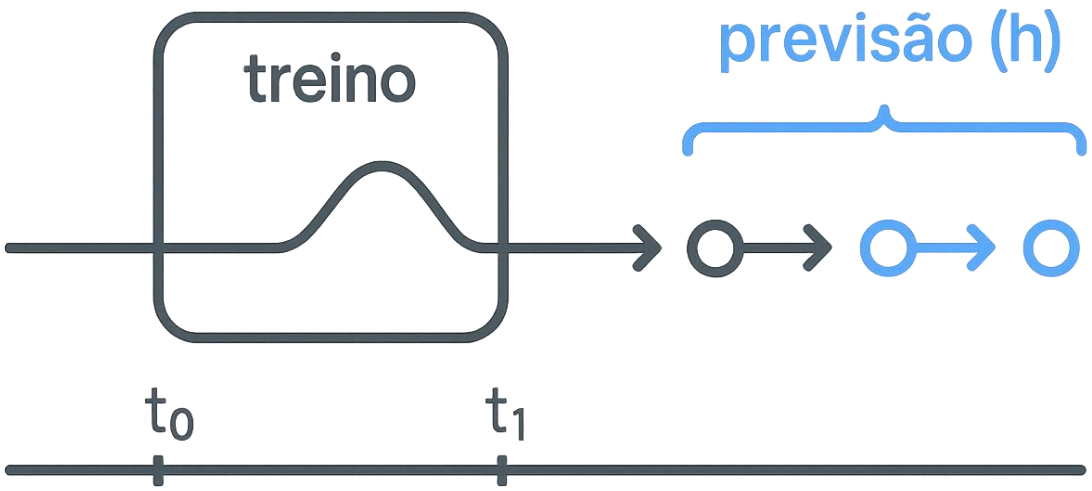
	Data	Real	Previsto	ErroAbs
0	2024-06-09	30.601663	30.599382	0.002281
1	2024-06-10	30.143467	32.129020	1.985552
2	2024-06-11	30.454893	30.821357	0.366465
3	2024-06-12	27.220571	28.448276	1.227705
4	2024-06-13	26.493874	27.007357	0.513483

Previsão:


Ajuste + Previsão (SARIMA selecionado)
Série
Ajuste (treino)
Previsão (h=10)
IC
Valor
34
32
30
28
26
24
2024-03-15
2024-04-01
2024-04-15
2024-05-01
2024-05-15
2024-06-01
2024-06-15
Data

Rolling Forecast

Rolling Forecast:



- É uma maneira de avaliar séries temporais em que o ponto de origem da previsão se movimenta ao longo do tempo

- Simula o uso real do modelo (sem vazamento de informação).
- Permite **medir desempenho multi-passos (h)** e a **estabilidade do modelo ao longo do tempo**.
- Evita vies de um único corte treino-teste “fixo”.

Rolling Forecast:

```python
def rolling_backtest(y, order, seas_order, h=7, initial=0.6):
    n0 = int(len(y)*initial)
    errors = []
    for end in range(n0, len(y)-h):
        train = y.iloc[:end]
        try:
            res = SARIMAX(train, order=order, seasonal_order=seas_order,
                          enforce_stationarity=False, enforce_invertibility=False
                         ).fit(disp=False, maxiter=100)
            fc = res.get_forecast(steps=h).predicted_mean.values
            err = np.abs(y.iloc[end:end+h].values - fc)
            errors.append(err)
        except:
            pass
    return np.mean(errors)
```

É importante não usar somente o melhor modelo, mas testar pelo menos os 5 com os menores BIC

Rolling Forecast: candidatos

	order	seasonal_order	BIC
0	(0, 1, 2)	(0, 1, 2, 7)	3131.608242
1	(0, 1, 2)	(1, 1, 2, 7)	3133.104160
2	(1, 1, 1)	(0, 1, 2, 7)	3135.923687
3	(0, 1, 1)	(0, 1, 2, 7)	3136.756582
4	(1, 1, 2)	(0, 1, 2, 7)	3137.313076
5	(1, 1, 2)	(1, 1, 2, 7)	3139.065061
6	(1, 1, 1)	(1, 1, 2, 7)	3139.544890
7	(0, 1, 1)	(1, 1, 2, 7)	3140.200690
8	(2, 1, 1)	(0, 1, 2, 7)	3140.577569
9	(1, 0, 2)	(0, 1, 2, 7)	3141.521071

Melhor por BIC: (0, 1, 2) x (0, 1, 2, 7) | BIC = 3131.61

Rolling Forecast: candidatos

	order	seasonal_order
0	(0, 1, 2)	(0, 1, 2, 7)
1	(0, 1, 2)	(1, 1, 2, 7)
2	(1, 1, 1)	(0, 1, 2, 7)
3	(0, 1, 1)	(0, 1, 2, 7)
4	(1, 1, 2)	(0, 1, 2, 7)
5	(1, 1, 2)	(1, 1, 2, 7)

```
MAE (rolling, h=10) vencedor): 1.2742283464410566
MAE (rolling, h=10) cand[1]: 1.2748362915496882
MAE (rolling, h=10) cand[2]: 1.2746984289771364
MAE (rolling, h=10) cand[3]: 1.2719077181246476
MAE (rolling, h=10) cand[4]: 1.2715331420828568
```

Instituto J&F

Centro de Educação para os Negócios